

Manual Oficial: MLflow Light

Visão Geral, Arquitetura, Setup, Guias de Uso (ML, LLM, Spans, Judge), Agentes de IA, Alta Concorrência e Códigos de Exemplo

Sumário

1. **Visão Geral e Repositório**
2. **Guia de Setup e Instalação**
3. **Arquitetura do Sistema**
4. **Guia de Uso: Modelos ML Tradicionais**
5. **Guia de Uso: LLMs e APIs (Gemini/OpenAI)**
6. **Guia de Uso: Spans e Árvores de Rastreamento (RAG/Agentes)**
7. **Guia de Uso: LLM as a Judge (Avaliações Automatizadas)**
8. **Instruções e Regras para Agentes Autônomos de IA**
9. **Guia de Alta Concorrência (Cloud Run/SQL)**
10. **Galeria de Códigos de Exemplo Prontos**

MLflow na Light — Remote Tracking Server no GCP

Repositório: https://github.com/Renanfavar01/MLflow_Light.git

Implementação completa do MLflow na Light utilizando a arquitetura **Remote Tracking with Server**, hospedado inteiramente no Google Cloud Platform.

O sistema atende a dois cenários principais da Light:

1. **Modelos de ML tradicionais** — treinamento, avaliação, versionamento de modelos (sklearn, XGBoost, etc.)
2. **Softwares com Foundation Models via API** — tracking de chamadas a LLMs (Gemini, OpenAI, etc.), prompts, tokens, latência e custos, incluindo frameworks de avaliação como LLM as a Judge e Spans para pipelines complexos (RAG, Agentes).

Estrutura do Repositório

- **infrastructure/**: Configurações do Terraform para provisionar Cloud SQL, Cloud Storage, Cloud Run, VPC, IAM, etc.
- **server/**: Container Docker customizado para rodar o MLflow Tracking Server conectado aos serviços GCP.
- **sdk/**: Pacote Python **light-mlflow** contendo decorators simplificados e integrações específicas para os times da Light.
- **sdk-node/**: Pacote Node.js (TypeScript/JS) contendo wrappers e funções para integração com aplicações web e backends JS.
- **examples/**: Exemplos práticos de uso do SDK e tracking de experimentos.
- **docs/**: Documentação detalhada sobre arquitetura, setup e guias de uso.

Autenticação

O acesso ao Tracking Server do MLflow é livre para a rede interna da Light. **Nenhuma autenticação (como IAP, Firebase ou Basic Auth) é exigida para os Cientistas de Dados** ou aplicações que logam as métricas via SDK. Toda a responsabilidade de acesso seguro aos recursos do GCP (Cloud SQL e Storage) fica a cargo exclusivo do MLflow Server via IAM (Service Accounts).

Requisitos para os Usuários (Projetos e Assistentes)

- Python 3.9+ ou Node.js (se usar o SDK JS)
- Git (para baixar o pacote)

Guia de Setup - MLflow Light

1. Instalando o SDK

Para que os cientistas de dados possam usar as funções do MLflow nos seus códigos locais ou nos notebooks (Jupyter/Colab):

```
pip install git+https://github.com/Renanfavarol/MLflow_Light.git#subdirectory=sdk
```

2. Configurando o Ambiente

Como o servidor MLflow está rodando na nuvem, você deve apontar o seu código para a URL oficial da Light:

Linux/Mac:

```
export MLFLOW_TRACKING_URI="https://mlflow-tracking-server-504082412074.us-central1.run.app"
```

Windows (PowerShell):

```
$env:MLFLOW_TRACKING_URI="https://mlflow-tracking-server-504082412074.us-central1.run.app"
```

Se estiver rodando o código sem essa variável configurada, os rastreamentos salvarão os arquivos na sua máquina local de forma indesejada.

Arquitetura do MLflow na Light

O MLflow na Light segue a arquitetura **Remote Tracking with Server** hospedado no GCP.

Componentes:

1. **Cloud Run (Server)**: Hospeda a UI e a API do MLflow. Configurado com acesso público para as redes permitidas, atuando como único gateway para o backend de metadados e armazenamento.
2. **Cloud SQL (PostgreSQL)**: Armazena os metadados (Runs, Parâmetros, Métricas, Tags). Não possui IP público e é acessado via Serverless VPC Access.
3. **Cloud Storage (GCS)**: Armazena os artefatos pesados (Arquivos `.pkl`, imagens, JSONs).

Segurança

- O Cloud Run não injeta a senha do banco em texto limpo nas variáveis de ambiente. Ele resolve a connection string diretamente do **Secret Manager**.
- O acesso de rede entre o Cloud Run e o Banco é feito inteiramente por tráfego interno do GCP (VPC).
- Todo o acesso aos recursos do GCP (GCS e Cloud SQL) é abstraído pelo Cloud Run. Os usuários e cientistas de dados interagem com a interface web ou com a API utilizando apenas a URL final, sem necessidade de autenticação (Firebase/IAP) ou contas GCP.

Guia Rápido: Modelos Tradicionais

Para modelos como Scikit-Learn e XGBoost, utilize o `@train_model`. Ele ativará o **autolog**, que captura hiperparâmetros (como `max_depth`, `n_estimators`), salva as métricas de treino e faz o upload automático do arquivo do modelo (`.pkl`) para o Cloud Storage.

```
from light_mlflow.decorators import train_model

@train_model(run_name="Treino_XGBoost")
def meu_treino():
    # Seu código normal...
    model.fit(X, y)
    return model
```

Guia Rápido: LLM Tracker Básico

Se você tem um script Python simples que faz chamadas diretas para a API do Gemini ou da OpenAI (sem usar LangChain), use o `@track_llm_call`.

Esse decorator salva:

- O modelo usado (`model_name`)
- A latência da chamada da API
- O prompt enviado (como texto nos artefatos)
- A resposta final gerada (como texto nos artefatos)

```
from light_mlflow.decorators import track_llm_call

@track_llm_call(model_name="gemini-1.5-pro")
def classificar_texto(prompt):
    # chamada para a API
    return resposta
```

Guia Rápido: Observabilidade Avançada de IA (Spans)

Quando construímos RAGs (Retrieval-Augmented Generation) ou Agentes, precisamos de **Tracing** em árvore, não apenas um log linear. O MLflow fornece isso de forma nativa e o SDK da Light facilita com decorators.

A regra de ouro é: **A função principal que desencadeia as outras deve ter o `@track_pipeline`.**

```
from light_mlflow.decorators import track_pipeline, retriever_span, llm_span

@retriever_span()
def buscar():
    pass

@llm_span()
def gerar():
    pass

@track_pipeline()
def main():
    docs = buscar()
    gerar(docs)
```

No painel do MLflow (na aba Traces), você verá o `main` como o pai, e `buscar` e `gerar` como filhos, com as respectivas latências.

Guia Rápido: LLM as a Judge

Para avaliar automaticamente a qualidade das respostas do seu Chatbot/RAG usando a própria inteligência artificial, use o `LLMJudge`.

1. Defina o seu avaliador (A IA que julga)

Você precisa criar uma função simples que chama a API (do Gemini, por exemplo) e pede pra ele dar uma nota de 1 a 5 baseada num critério (Relevância, Coerência).

```
def gemini_avalizador(input_text, output_text, context):  
    # Faz o request para o Gemini pedindo a nota  
    return {"score": 5, "justification": "Resposta perfeita e clara."}
```

2. Aplique o Decorator

Importe a classe e coloque o decorator `@judge_output` na sua função principal do Chatbot:

```
from light_mlflow.evaluation import LLMJudge, JudgeCriteria, judge_output  
  
juiz = LLMJudge(evaluator_func=gemini_avalizador)  
  
@judge_output(juiz, criterion=JudgeCriteria.RELEVANCE)  
def meu_chatbot(prompt):  
    return "Resposta do meu bot..."
```

O MLflow criará automaticamente um Span chamado `llm_judge_relevance`, salvará a nota em um gráfico e gravará a justificativa como um arquivo `.json`.

Integração Light MLflow para Agentes Autônomos

Objetivo

Este guia serve como a principal base de conhecimento para o Antigravity (ou outras IAs) aprenderem a integrar o SDK padrão de Observabilidade da Light (`light_mlflow`) em novos projetos de Agentes de IA, RAGs e orquestradores.

1. Dependência e Instalação

Ao criar ou atualizar um projeto Python, o pacote do MLflow da Light deve ser inserido no arquivo `requirements.txt` utilizando o repositório Git:

```
# Adicionar no requirements.txt  
light-mlflow @  
git+https://github.com/Renanfavarol1/MLflow_Light.git#subdirectory=sdk
```

2. Configuração no Código (Startup)

Em todo novo projeto (seja FastAPI, Flask, ou scripts batch), o MLflow deve ser inicializado **uma única vez** no startup da aplicação, antes de qualquer execução de IA:

```
from light_mlflow import LightMLflowConfig
```

```
# A URI de conexão com o banco/servidor será injetada automaticamente
# pelas variáveis de ambiente do Cloud Run (MLFLOW_TRACKING_URI).
LightMLflowConfig.setup(experiment_name="Nome_do_Projeto_Novo")
```

🔍 3. Rastreamento Nativo (Traces) para Agentes

Para fluxos agênticos e RAGs, a IA desenvolvedora deve utilizar **EXCLUSIVAMENTE** a arquitetura de **spans** baseada em decoradores. Isso fará com que o painel do MLflow gere uma árvore visual detalhada (Traces) mostrando latência, prompts e ferramentas acionadas.

Importação Padrão:

```
from light_mlflow.decorators import track_pipeline, agent_span, tool_span,
llm_span, retriever_span
```

Regras Estritas de Aplicação (Para a IA Desenvolvedora)

1. **@track_pipeline(run_name="...")**: DEVE ser colocado apenas na **função principal** ou endpoint de entrada (ex: `chat()`, `main()`). Ele é o orquestrador que abre a gravação. Não use `mlflow.start_run()` manualmente.
2. **@retriever_span(name="...")**: Obrigatório em funções de busca (Elasticsearch, Pinecone, buscas em banco de dados para RAG).
3. **@llm_span(name="...")**: Obrigatório nas funções que enviam o prompt e recebem a resposta do provedor de IA (Gemini, VertexAI, OpenAI).
4. **@tool_span(name="...")**: Obrigatório em qualquer ferramenta consumida pelo Agente (ex: `consultar_saldo_sap`, `buscar_clima`).
5. **@agent_span(name="...")**: Opcional, usado para encapsular a lógica de raciocínio interno/loops de agentes ReAct.

Exemplo Arquitetural a Ser Seguido

```
from light_mlflow.decorators import track_pipeline, tool_span, llm_span

@tool_span(name="Consultar_Faturas_SAP")
def buscar_faturas(cpf):
    # Lógica de integração externa
    return f"Faturas abertas para {cpf}: R$ 150,00"

@llm_span(name="Gemini_Gerador_Resposta")
def gerar_resposta_ao_cliente(contexto, pergunta):
    # Inferência LLM
    return f"Baseado no sistema: {contexto}"

@track_pipeline(run_name="Atendimento_Cliente_Fluxo")
def processar_mensagem(pergunta, cpf):
    contexto = buscar_faturas(cpf)
```

```
resposta = gerar_resposta_ao_cliente(contexto, pergunta)
return resposta
```

🚨 Atenção (Avisos Críticos de Arquitetura)

- **Não misturar abordagens:** NUNCA utilize o antigo `@track_llm_call` em projetos com Agentes, pois ele não gera a árvore hierárquica na aba Traces.
- **Evite hardcode de URLs:** Nunca defina a `tracking_uri` manualmente no código. Deixe o SDK ler da variável de ambiente gerenciada pelo Terraform/Cloud Run.
- **Segurança de SSL:** Em caso de avisos de `InsecureRequestWarning` devido à rede interna da Light, não desative os warnings no código do agente (deixe isso para a camada de infra/Docker se necessário, ou garanta que o CA corporativo esteja instalado).

🌀 4. Integração Node.js / TypeScript (Apenas Backend)

Se o projeto de destino for em Node.js (ex: Express, NestJS, Vite SSR), o MLflow deve ser integrado usando o pacote NPM centralizado da Light. O SDK Javascript utiliza *Wrappers* e *AsyncLocalStorage* em vez de decoradores.

Instalação (package.json):

```
"dependencies": {
  "light-mlflow-node":
  "git+https://github.com/Renanfavarol1/MLflow_Light.git#subdirectory=sdk-node"
}
```

Configuração Inicial (Index.js / Main.js)

```
import { LightMLflowConfig } from 'light-mlflow-node';
// A URI é lida automaticamente do process.env.MLFLOW_TRACKING_URI
await LightMLflowConfig.setup("Nome_do_Projeto_Node");
```

Rastreamento de LLMs em Node.js

Use as funções `trackPipeline` e `llmSpan` para englobar a lógica. A extração de tokens é automática se a função retornar objetos da OpenAI ou `@google/genai` (Node.js).

```
import { trackPipeline, llmSpan } from 'light-mlflow-node';

const minhaChamadaGemini = llmSpan("Gemini_Resumo", async (texto) => {
  // Integração com o SDK do Google
  return response;
});
```

```
const processarChamada = trackPipeline("Atendimento_Node", async (req) => {
  const dados = await minhaChamadaGemini(req.body.pergunta);
  return dados;
});
```

Guia de Alta Concorrência: Assistente Virtual + MLflow

Este guia destina-se a aplicações de altíssimo tráfego (como o Assistente Virtual rodando em FastAPI), onde milhares de chamadas de inferência de LLM ou chamadas de ferramentas ocorrem concorrentemente.

O Desafio

Os decoradores padrões do SDK (`@llm_span`, `@track_pipeline`) são projetados para simplicidade e **bloqueiam a thread** enquanto enviam os dados de telemetria para o servidor remoto do MLflow via HTTP. Em um cenário de mil requisições simultâneas, esperar o retorno da API do MLflow pode aumentar o tempo de resposta do chatbot (latência) ou gerar gargalos no seu event loop.

A Solução: FastAPI BackgroundTasks

A melhor prática para não impactar o usuário final do Assistente Virtual é executar a cadeia de raciocínio (ou rastreamento principal) e delegar o envio ao MLflow para o background de forma assíncrona. Felizmente, no Cloud Run com CPU "Always On" (padrão em aplicações de alto tráfego) e no FastAPI, as `BackgroundTasks` são perfeitas para isso.

Exemplo Arquitetural

```
from fastapi import FastAPI, BackgroundTasks
from light_mlflow.decorators import track_pipeline, llm_span
from light_mlflow import LightMLflowConfig
import asyncio

app = FastAPI()
LightMLflowConfig.setup(experiment_name="Assistente_Virtual_Producao")

# 1. Suas lógicas de negócio ficam isoladas, porém instrumentadas:
@llm_span(name="Gerador_Resposta_Gemini")
async def gerar_resposta(pergunta: str):
    # Simula chamada a API da OpenAI/Google
    await asyncio.sleep(1)
    return "Resposta processada!"

@track_pipeline(run_name="Interacao_Usuario_Chatbot")
async def fluxo_principal(pergunta: str):
    # Essa função orquestra as tools e o LLM e será rastreada.
    resposta = await gerar_resposta(pergunta)
    return resposta

# 2. O Endpoint do FastAPI apenas enfileira o rastreamento!
```



```
@app.post("/chat")
async def chat_endpoint(pergunta: str, bg_tasks: BackgroundTasks):

    # Executar a lógica dentro do ciclo de background (forma fire-and-forget
    # controlada)
    # NOTA: O FastAPI só enviará o processamento após responder o JSON 200 pro
    # usuário.
    # No entanto, caso você precise do resultado do LLM *para* o usuário na mesma
    # hora,
    # você processa o LLM na frente, e faz o log de metadados em background (Opção
    # Avançada).

    bg_tasks.add_task(fluxo_principal, pergunta)
    return {"status": "Processamento iniciado no background."}
```

Tolerância a Falhas do SDK

O SDK `light_mlflow` já está configurado para efetuar múltiplas retentativas exponenciais (`MLFLOW_HTTP_REQUEST_MAX_RETRIES=10`) caso o servidor do MLflow esteja sobrecarregado (HTTP 429), garantindo que seu dado não se perca durante um pico extremo de conexões.

Exemplos de Uso - Light MLflow SDK

Esta pasta contém exemplos práticos de como utilizar as ferramentas do SDK `light-mlflow`.

1. `ml_model_example.py`: Mostra como rastrear o treinamento de modelos tradicionais (Sklearn/XGBoost) com autolog.
2. `rag_pipeline_example.py`: Demonstra o uso de `Spans` aninhados para criar árvores de rastreamento para pipelines GenAI complexos.
3. `llm_judge_example.py`: Mostra como implementar o padrão "LLM as a Judge" para avaliar respostas de LLMs automaticamente usando o nosso decorator.

Para rodar qualquer um, certifique-se de ter instalado o SDK localmente (`pip install -e ./sdk`).

Galeria de Códigos de Exemplo

Abaixo estão os scripts de referência para você copiar e colar nos seus projetos.

agent_example.py

```
from light_mlflow import LightMLflowConfig
from light_mlflow.decorators import track_pipeline, agent_span, tool_span,
llm_span
import time

LightMLflowConfig.setup(experiment_name="Light_Agente_Resolucao")
```

```

@tool_span(name="Consultar_Fatura_API")
def buscar_fatura(cpf):
    time.sleep(0.3)
    return {"status": "atrasada", "valor": 150.00}

@llm_span(name="Gemini_Decisao")
def gemini_decidir_acao(contexto):
    time.sleep(1)
    if contexto["status"] == "atrasada":
        return "Gerar código de barras"
    return "Avisar que está tudo certo"

@agent_span(name="Agente_Financeiro")
def agente_loop(cpf):
    fatura = buscar_fatura(cpf)
    decisao = gemini_decidir_acao(fatura)
    return decisao

@track_pipeline(run_name="Atendimento_Agente_Autonomo")
def chat_agente(cpf):
    resultado = agente_loop(cpf)
    return resultado

if __name__ == "__main__":
    print(chat_agente("123.456.789-00"))

```

llm_api_example.py

```

from light_mlflow import LightMLflowConfig
from light_mlflow.decorators import track_llm_call
import time

LightMLflowConfig.setup(experiment_name="Light_LLM_APIs")

# O decorator registra o prompt, o tempo de resposta, e a resposta final.
@track_llm_call(model_name="gemini-2.5-flash", track_prompt=True)
def classificar_sentimento(prompt):
    print("Chamando a API do Gemini...")
    time.sleep(1) # Simula a chamada de rede

    if "ruim" in prompt.lower():
        return "NEGATIVO"
    return "POSITIVO"

if __name__ == "__main__":
    resultado = classificar_sentimento(prompt="O atendimento hoje foi muito ruim, a luz não voltou.")

```

```
print(f"Resultado: {resultado}")
```

llm_judge_example.py

```
from light_mlflow import LightMLflowConfig
from light_mlflow.evaluation import LLMJudge, JudgeCriteria, judge_output

LightMLflowConfig.setup(experiment_name="Avaliacao_Automatica")

def meu_avaliador_gemini(input_text, output_text, context):
    """
    Aqui você faria a chamada real para a API do Gemini, pedindo para ele dar
    uma nota de 1 a 5 baseado no critério.
    """
    # Simulando a resposta do Gemini:
    return {
        "score": 4,
        "justification": "A resposta é boa, mas não cita a segunda via pelo site."
    }

# Inicializa a classe do Juiz
juiz = LLMJudge(evaluator_func=meu_avaliador_gemini)

# Aplica o decorator na função do seu chatbot
@judge_output(juiz, criterion=JudgeCriteria.COMPLETENESS)
def responder_cliente(prompt):
    return "Você pode pedir a segunda via pelo App da Light."

if __name__ == "__main__":
    # Ao executar isso, o MLflow registrará o prompt, a resposta e, em seguida,
    # chamará o juiz para dar a nota e salvará no painel.
    responder_cliente(prompt="Como peço segunda via?")
```

ml_model_example.py

```
from light_mlflow import LightMLflowConfig
from light_mlflow.decorators import train_model
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 1. Configura a conexão (usa MLFLOW_TRACKING_URI se não passar a URI)
LightMLflowConfig.setup(experiment_name="Previsao_de_Fraude")

# 2. O decorator ativa o autolog() do MLflow automaticamente
```

```
@train_model(run_name="RandomForest_Baseline")
def executar_treinamento():
    X, y = load_iris(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    model = RandomForestClassifier(n_estimators=50, max_depth=5)

    # Neste momento, o MLflow captura os hiperparâmetros e salva o modelo no GCP
    model.fit(X_train, y_train)

    score = model.score(X_test, y_test)
    print(f"Treinamento concluído. Acurácia: {score}")

    return model

if __name__ == "__main__":
    executar_treinamento()
```

node_example.mjs

```
import { LightMLflowConfig, trackPipeline, llmSpan } from '../sdk-
node/src/index.js';

// Simulando a variável de ambiente (geralmente injetada pelo Cloud Run)
process.env.MLFLOW_TRACKING_URI = "https://mlflow-tracking-server-lvvvhziq5a-
uc.a.run.app";

// 1. Inicializa (deve ser feito apenas 1 vez na inicialização da aplicação)
await LightMLflowConfig.setup("Light_Node_APIs");

// 2. Simula uma chamada à LLM que retorna uso de tokens (estilo SDK do Google
GenAI para Node)
const gerarRespostaLLM = llmSpan("Gemini_NodeJS", async (prompt) => {
    console.log("Chamando a API do Gemini...");
    // Simulando delay de rede
    await new Promise(r => setTimeout(r, 1000));

    // Simulando objeto de resposta da API do Google GenAI (node)
    return {
        text: "Resposta simulada",
        usageMetadata: {
            promptTokenCount: 120,
            candidatesTokenCount: 50,
            totalTokenCount: 170
        }
    };
});

// 3. Orquestrador Principal (Endpoint do Backend)
```

```
const processarMensagem = trackPipeline("Atendimento_Cliente", async (pergunta) =>
{
    const resposta = await gerarRespostaLLM(pergunta);
    return resposta;
});

// 4. Teste de execução
console.log("Iniciando processamento...");
const resultado = await processarMensagem("Olá, minha conta de luz está muito cara.");
console.log("Concluído. Verifique o painel do MLflow na aba 'Metrics' do experimento 'Light_Node_APIs'.");
```

rag_pipeline_example.py

```
import time
from light_mlflow import LightMLflowConfig
from light_mlflow.decorators import track_pipeline, retriever_span, llm_span

LightMLflowConfig.setup(experiment_name="Light_RAG_Atendimento")

@retriever_span(name="Busca_Base_Conhecimento")
def buscar_documentos(pergunta):
    time.sleep(0.5) # Simulando busca no Elasticsearch
    return ["A Light atende a zona sul do RJ", "Segunda via pode ser pedida no app"]

@llm_span(name="Chamada_Gemini_Pro")
def gerar_resposta(pergunta, contexto):
    time.sleep(1.0) # Simulando geração de texto
    return f"De acordo com a base, a Light atende a zona sul."

# O track_pipeline abraça todos os passos abaixo gerando uma árvore visual
@track_pipeline(run_name="Pergunta_Cliente")
def pipeline_principal(pergunta):
    contexto = buscar_documentos(pergunta)
    resposta = gerar_resposta(pergunta, contexto)
    return resposta

if __name__ == "__main__":
    resposta_final = pipeline_principal("Onde a Light atua?")
    print("Resposta:", resposta_final)
```

test_trace.mjs

```
import { LightMLflowConfig, trackPipeline, llmSpan, toolSpan } from '../sdk-
node/src/index.js';

// Simulando a variável de ambiente (geralmente injetada pelo Cloud Run)
process.env.MLFLOW_TRACKING_URI = "https://mlflow-tracking-server-lvvvhziq5a-
uc.a.run.app";

console.log("Iniciando Setup...");
await LightMLflowConfig.setup("Light_Node_Traces_Test");

const mockTool = toolSpan("Buscar_Dados_SAP", async (id) => {
  console.log(`[Tool] Buscando dados para id: ${id}...`);
  await new Promise(r => setTimeout(r, 500));
  return { status: "success", data: "Cliente SAP OK" };
});

const mockLLM = llmSpan("Gemini_Mock", async (prompt, context) => {
  console.log("[LLM] Gerando resposta...");
  await new Promise(r => setTimeout(r, 1000));
  return {
    text: `Resposta para: ${prompt}. Contexto: ${JSON.stringify(context)}`,
    usageMetadata: {
      promptTokenCount: 150,
      candidatesTokenCount: 80,
      totalTokenCount: 230
    }
  };
});

const processarMensagem = trackPipeline("Atendimento_Teste_Trace", async
(pergunta, userId) => {
  const sapData = await mockTool(userId);
  const resposta = await mockLLM(pergunta, sapData);
  return resposta;
});

console.log("Iniciando Pipeline...");
const res = await processarMensagem("Como está minha fatura?", "12345");
console.log("Resultado final:", res);

console.log("Aguardando 6s para flush dos traces...");
await new Promise(r => setTimeout(r, 6000));
console.log("Teste finalizado. Verifique a aba Traces no MLflow!");
```